Logbook: Efficient ResNet for CIFAR-10 (5M parameter limit)

This logbook records all the experiments that we did and all the results that we got, in time order. The language follows ASD-STE100 Simplified Technical English. Each sentence gives one idea.

The repository contains only the code for the final model (Section 8). The code for the other experiments is not in the repository. The configuration and the training history of each run are in `results/runs/<run>/`.

1. Task and rules

The task is a Kaggle competition for a course. The rules are:

1. Train a modified ResNet. The network must have skip connections.
2. The model must have a maximum of 5,000,000 trainable parameters.
3. Train from zero. Do not use pre-trained weights.
4. Use only the CIFAR-10 training data. Do not use other datasets.
5. Do not label the test images by hand. Do not train on the test images.
6. You can use optimizers, augmentation, regularization, schedules and distillation.
7. The competition closes on 2026-11-06 at 23:59. The report is due on 2026-11-09.

The metric is classification accuracy. Kaggle shows a public score on approximately half of the hidden images. The private score on the other half sets the final rank.

2. Data

The `data/` folder contains these files:

File	Contents
`train_labels.csv`	50,000 rows: `id`, `filename`, `label` (0 to 9)
`train_images/`	50,000 RGB PNG images, 32 × 32 pixels
`test.csv`	2,000 rows: `id`, `filename`
`test_images/`	2,000 RGB PNG images, 32 × 32 pixels
`sample_submission.csv`	2,000 rows: `id`, `label`

Each class has exactly 5,000 training images. The test IDs are 0 to 1999. They are in the same order as the sample submission.

`src/prepare_data.py` decodes all PNG files one time. It writes `data/cifar10_cache.npz`. The channel means of the training images are 0.4914, 0.4822 and 0.4465. These are the standard CIFAR-10 values.

The test set has 2,000 images. This is the same size as CIFAR-10.1 (Recht et al., 2018). CIFAR-10.1 is a new test set with a small distribution shift from CIFAR-10. Models usually get 4 to 10 points less accuracy on CIFAR-10.1 than on CIFAR-10. Our Kaggle scores agree with this. Thus, we think that the hidden set is similar to CIFAR-10.1.

3. Method for evaluation

We did not select models with the public Kaggle score. The public split has only approximately 1,000 images. One score has a standard error of approximately 0.9 points. Selection on that split can overfit.

We use a holdout set from the training data. The holdout has 5,000 images: 500 from each class. A fixed seed makes the split. The model trains on the other 45,000 images. All runs in this logbook use the same holdout, except the seed-1 runs in Phase 3 and Phase 5.

All holdout accuracies use test-time augmentation (TTA), unless the text gives a different method. TTA averages the outputs for the image, its mirror, and two 1-pixel shifts with their mirrors. For SLA models, “head” is the 10-class head, “vote” is the 4-rotation vote, and “head + vote” is the mean of the two softmax outputs. We submit head + vote.

For a submission, we train the selected recipe again on all 50,000 images. That model has no holdout.

Rule for a change: we keep a change only if it adds at least 0.3 points on the holdout. From Phase 9, we also give the number of holdout images that a change fixes and breaks, with a paired McNemar test.

4. Hardware

Hardware	Use	Speed
Apple M4 Pro, MPS backend	Phase 1 and Phase 2, prediction	Approximately 23 s per epoch for the 4.8M AirResNet
NVIDIA A100 40 GB, NYU HPC	Phase 3 to Phase 10	Approximately 6.6 s per epoch for the 4.7M PreActResNet

All data stays on the GPU. The GPU does the crop, flip and cutout operations. TrivialAugment runs on CPU worker processes. The workers prepare the next epoch during training. HPC jobs stop after 3 to 5 hours. Thus, all long runs save a checkpoint every 10 epochs and can continue after a stop.

5. Literature research

We did three literature surveys. The reports, search logs and BibTeX files are in `output/literature-survey/`. `docs/RESEARCH_REPORT.md` gives the ranked plan.

Main results from the research:

1. Accuracy on a shifted test set follows accuracy on the original test set on a straight line. For CIFAR-10.1: new accuracy = 1.62 × CIFAR-10 accuracy − 65.51 (Recht et al., 2018). Thus, each +1 point on the holdout gives approximately +1.6 points on the hidden set.
2. Almost no method moves a model above this line without more training data (Taori et al., 2020).
3. Cutout and wider networks help more on CIFAR-10.1 than on CIFAR-10.
4. Models that train only on CIFAR-10 are 0.1 to 1.7 points below the line (Shi et al., 2023).
5. RegMixup (Pinto et al., 2022) gives a good result on CIFAR-10.1 for a WideResNet: 97.46% clean, 92.79% on CIFAR-10.1.

6. Experiments in time order

Phase 1: AirResNet on the M4 Pro (2026-10-29)

The first model is “AirResNet”. It comes from airbench96 (Jordan, 2024). It has a frozen 2×2 whitening layer, three stages of conv → max-pool → BN → GELU, and residual pairs of 3×3 convs. The recipe uses Nesterov SGD, batch 1024, label smoothing 0.2, flip, 4-pixel translation, 12-pixel cutout and a Lookahead weight average.

We tested three width and depth settings for 40 epochs:

Run	Widths	Residual pairs	Params	Holdout, one view	Holdout, TTA	Time
`w128-256-352_d111_e40`	128/256/352	1/1/1	4.84M	94.90%	**95.58%**	13.6 min
`w96-256-352_d211_e40`	96/256/352	2/1/1	4.80M	95.06%	95.36%	13.2 min
`w96-192-384_d111_e40`	96/192/384	1/1/1	4.34M	94.60%	95.10%	9.1 min

The widest model was the best. Its Kaggle public score was **0.850**.

Phase 2: longer training and TrivialAugment (2026-10-29)

TrivialAugment (Müller and Hutter, 2021) applies one random image operation to each image before flip, translation and cutout. It does not increase the epoch time.

Run	Epochs	TrivialAugment	Holdout	Time	Kaggle
`w128-256-352_d111_e40`	40	no	95.58%	13.6 min	0.850
`w128-256-352_ta_e40`	40	yes	94.98%	13.6 min	
`w128-256-352_e120`	120	no	96.02%	40.6 min	
`w128-256-352_ta_e120`	120	yes	**96.20%**	40.9 min	0.860

Longer training added 0.4 to 0.6 points. TrivialAugment decreased the accuracy at 40 epochs. It added 0.2 points at 120 epochs. Thus, TrivialAugment needs a long schedule.

Diagnostic: we changed the holdout images to the brightness and contrast of the test images. We used only the pixel statistics of the test images. We did not classify the test images.

Model	Clean	Test statistics × 1	Test statistics × 2	Brightness spread +0.07	Contrast × 1.15
`w128-256-352_d111_e40`	95.58	95.46	95.16	95.42	95.36
`w96-256-352_d211_e40`	95.36	95.14	95.08	95.06	95.04
`w128-256-352_e120`	96.02	95.94	95.68	95.78	95.78
`w128-256-352_ta_e120`	96.20	96.16	96.16	95.96	96.00

The accuracy decreased by a maximum of 0.4 points. Thus, brightness and contrast do not cause the Kaggle gap.

Two short tests on the Mac (2 epochs each):

- SAM (Foret et al., 2021) took 46 s per epoch. Plain SGD took 23 s per epoch.
- SLA at batch 1024 did not fit in the 24 GB memory of the M4 Pro. At batch 512, it took 95 s per epoch.

Thus, we moved the next phases to the A100.

Phase 3: rotation self-supervision (SLA) on the A100 (2026-10-29)

SLA (Lee et al., 2020) trains the network on the 4 rotations of each image. A joint head has 40 outputs (class × rotation). The label of an image that turned by k × 90 degrees is class × 4 + k. A 10-class head learns from the averaged rotation prediction (self-distillation, "SLA+SD"). At test time, we use the 10-class head, the 4-rotation vote, or the mean of the two. We compared our code with the paper and with the official code (hankook/SLA).

AirResNet 128/256/352, TrivialAugment, 120 epochs, batch 1024. The control is the same recipe without SLA on the same GPU.

Run	Seed	Holdout (head / vote / head + vote)	Time	Control without SLA	Kaggle
`sla_ta_e120_b1024`	0	96.30% / 96.32% / **96.40%**	12.4 min	95.90% (`ta_e120_b1024_a100`, 4.0 min)	0.863
`sla_ta_e120_b1024_s1`	1	**96.68%** / 96.30% / –		96.28% (`ta_e120_b1024_s1`)	0.857 (control)

SLA added 0.4 to 0.5 points on two seeds. But all AirResNet models stayed near 0.86 on Kaggle.

Phase 4: standard pre-activation ResNet (2026-10-29)

All AirResNet models were approximately 4 points below the CIFAR-10.1 line. Thus, we changed the architecture family.

The new model is a standard pre-activation ResNet (He et al., 2016):

- A 3×3 stem conv with 64 channels.
- Three stages with 4, 4 and 3 basic blocks. The widths are 64, 128 and 256.
- Each block is BN → ReLU → conv 3×3 → BN → ReLU → conv 3×3, plus a skip connection.
- The skip is the identity, or a 1×1 conv when the shape changes.
- Global average pooling and one linear layer.

The recipe: Nesterov SGD, lr 0.4 at batch 512, weight decay 5e-4, 5 warmup epochs, cosine schedule, label smoothing 0.1, TrivialAugment, flip, 4-pixel translation, 12-pixel cutout, 200 epochs.

Run	Params	Holdout	Time	Kaggle	Recht line
`preact_ta_e200`	4,696,394	96.82%	22 min	**0.884**	91.3%

The holdout gain was 0.4 points. The Kaggle gain was 2.1 points. The AirResNet models were approximately 4 points below the line. The PreActResNet was approximately 3 points below. Thus, the architecture family has an effect on the shift.

Phase 5: PreActResNet + SLA (2026-10-29 and 2026-10-30)

Run	Data	Holdout (head / vote / head + vote)	Time	Kaggle
`preact_sla_ta_e200`	45,000	**97.56%** / 97.42% / 97.54%	85 min	0.903
`preact_sla_ta_e200_full`	50,000	–	92 min	**0.909**
`preact_sla_ta_e200_s1`	45,000 (seed 1, own split)	97.78% / 97.54% / –	83 min	

SLA added 0.74 points on the PreActResNet. The difference between the two seeds is approximately 0.2 points. The 50k model agrees with the 45k model on 96.2% of the test images. This recipe stayed our submission until Phase 10.

Phase 6: other methods from the research report (2026-10-30)

We tested each method against the correct baseline. No method was better than PreActResNet + SLA.

Run	Method	Holdout	Baseline	Time	Result
`preact_ta_e200_sam05`	SAM, ρ = 0.05 (Foret et al., 2021)	97.18%	96.82%	44 min	+0.36
`preact_sla_ta_e200_sam05`	SAM + SLA	97.58% (head + vote)	97.54%	2.8 h	+0.04, not kept (2× cost)
`papa4_preact_ta_e200`	PAPA, 4 copies (Jolicoeur-Martineau et al., 2024)	97.18–97.20% per copy	96.82%	103 min	below 97.56%
`shake2x40d_ta_e600`	Shake-Shake-26 2x40d, 600 epochs (Gastaldi, 2017)	97.16%	97.56%	1.8 h	below
`preact_blur_ta_e200`	Anti-aliased downsampling (Zhang, 2019)	97.00%	96.82%	25 min	+0.18, not kept
`preact_ta_e400`	400 epochs	97.12%	96.82%	44 min	+0.30, below 97.56%
`preact_kd_sla2_ta_e200`	Distillation from 2 SLA teachers (Hinton et al., 2015)	97.18%	97.56%	33 min	below

Notes:

- SAM helps the plain ResNet. SAM does not help on top of SLA.
- In the PAPA run, the averaged model got only 96.20%. The cause was a bug: the code calculated the BN statistics again on clean images. With BN statistics from augmented images, the averaged model got 96.94%.
- Shake-Shake needs a long schedule. We trained it for 600 epochs over two HPC sessions.
- Distillation: the student is a PreActResNet without SLA. The teachers are `preact_sla_ta_e200` and `preact_sla_ta_e200_sam05`. They trained on the same 45,000 images. Thus, the holdout stays clean.

The teachers and the student see the same augmented image. The loss is 0.5 × cross-entropy + 0.5 × KL divergence at temperature 4. The student got +0.36 points over the same ResNet without distillation. But it is 0.38 points below the teachers' recipe.

Phase 7: CutMix inside SLA (2026-10-31)

`preact_sla_cutmix_e200`: PreActResNet + SLA with CutMix (Yun et al., 2019). With probability 0.5, each batch gets one box from a shuffled partner image. The box goes in before the four rotations. The losses are mixed by area. The run also kept a weight average (EMA, decay 0.999). Time: 84 min.

Prediction	CutMix, 200 epochs	PreActResNet + SLA (seed 0)
Head	97.42%	97.56%
Vote	97.52%	97.42%
EMA, head	96.98%	–

CutMix did not add accuracy. The EMA was 0.4 points below the live network. Section 10 gives the cause.

Phase 8: CutMix for 300 epochs, per-copy augmentation and 300 epochs (2026-10-31)

All runs use seed 0 and 45,000 images.

- `preact_sla_cutmix_e300`: the CutMix run for 300 epochs, without the EMA.
- `preact_sla_percopy_e200`: each of the four rotated copies of an image gets its own random crop, flip and cutout.
- `preact_sla_ta_e300`: PreActResNet + SLA for 300 epochs, without CutMix. This run is the control for the CutMix run.

Prediction	Per-copy, 200 epochs	300 epochs	CutMix, 300 epochs	SLA, 200 epochs
Head	97.58%	97.68%	97.66%	97.56%
Vote	97.52%	97.68%	97.64%	97.42%
Head + vote	97.66%	97.74%	97.70%	97.54%
Time	83 min	125 min	124 min	85 min

Per-copy augmentation did not change the accuracy. Thus, we did not do a two-view consistency run. CutMix did not add accuracy at 200 or at 300 epochs. On head + vote, 300 epochs added +0.20 points. This is less than +0.3 points, but it is the largest gain. Thus, we trained it again on all 50,000 images: `preact_sla_ta_e300_full` (139 min, Kaggle 0.906).

Phase 9: wider and shallower network (2026-11-01)

`preact_wide_sla_ta_e200`: a PreActResNet with widths 80/160/320 and 3/3/2 blocks per stage. It has 4,926,210 parameters with the 40-class head. The recipe is SLA, TrivialAugment, cutout, 200 epochs, batch 512. The run also kept an EMA (decay 0.999), with the BN fix of Section 10. Time: 89 min.

Prediction	Wide net, 200 epochs	SLA, 200 epochs	SLA, 300 epochs
Head	97.74%	97.56%	97.68%
Vote	97.54%	97.42%	97.68%
Head + vote	97.76%	97.54%	97.74%

Prediction	Wide net, 200 epochs	SLA, 200 epochs	SLA, 300 epochs
EMA, head + vote	97.70%	–	–

Comparison (head + vote)	Images fixed	Images broken	McNemar p
Wide net vs SLA, 200 epochs	38	27	0.21
Wide net vs SLA, 300 epochs	31	30	1.0

The wide net is equal to SLA-300 on the holdout. The EMA did not help. We trained the wide net again on all 50,000 images (`preact_wide_sla_ta_e200_full`, 98 min). On Kaggle, both wide-net models are below the narrow models (Section 7). Thus, we did not keep the wide net.

Phase 10: RegMixup inside SLA (2026-11-02 and 2026-11-03)

RegMixup (Pinto et al., 2022) adds a mixed copy of each batch to the clean batch. One λ from Beta(20, 20) mixes each image with a shuffled partner image. The mixed copy takes both labels with the weights λ and 1 – λ, on the joint head and on the 10-class head. Self-distillation stays on the clean images. The mixed loss has the weight 1. The mixed copy goes in before the four rotations. The batch is 256 clean images + 256 mixed images (2,048 rotated images per step, as in the baseline). The lr is 0.2. On the A100, one epoch takes approximately 50 s.

Before the first run, we tested the loss with 7 checks and with two short runs on the HPC node.

Run	Data	Epochs	Holdout (head / vote / head + vote)	Time	Kaggle
`preact_sla_regmix_e200`	45,000	200	97.92% / – / 97.86%	2.8 h	0.907
`preact_sla_regmix_e200_full`	50,000	200	–	3.1 h	0.904
`preact_sla_regmix_e300`	45,000	300	98.00% / 97.58% / **97.94%**	4.2 h	
`preact_sla_regmix_e300_full`	50,000	300	–	4.6 h	**0.912**

Paired comparisons on the holdout (head + vote):

Baseline	Baseline	New run	New run	Fixed	Broken	Net	McNemar p
`preact_sla_ta_e200`	97.54%	RegMixup-200	97.86%	47	31	+16 (+0.32)	0.089
`preact_sla_ta_e300`	97.74%	RegMixup-200	97.86%	38	32	+6 (+0.12)	0.550
`preact_sla_regmix_e200`	97.86%	RegMixup-300	97.94%	28	24	+4 (+0.08)	0.678
`preact_sla_ta_e300`	97.74%	RegMixup-300	97.94%	43	33	+10 (+0.20)	0.302
`preact_sla_ta_e200`	97.54%	RegMixup-300	97.94%	53	33	+20 (+0.40)	0.040

RegMixup-200 passes the 0.3-point rule against its baseline. RegMixup-300 has the best holdout accuracy of all runs. Only the comparison with SLA-200 is significant at 0.05. There is one confound: the baseline

used batch 512. Batch 512 clean + 512 mixed does not fit in 40 GB.

We also calculated an ensemble of 4 teachers (SLA-200, SLA-300, the wide net, RegMixup-200) on the holdout: 97.92%. The best single model is equal to this ensemble. Thus, distillation into one network can give only a small gain. We did not do a distillation run.

The 50k RegMixup-300 model agrees with the 0.909 model on 95.3% of the test images. Its Kaggle public score is 0.912. This is the best public score. The difference to 0.909 is approximately 3 public images. This is in the noise. We selected RegMixup-300 on the holdout (97.94%), not on the public score.

7. Kaggle public scores

#	Run	Model	Holdout	Kaggle public
1	`w128-256-352_d111_e40`	AirResNet, 40 epochs	95.58%	0.850
2	`w128-256-352_ta_e120`	AirResNet + TrivialAugment, 120 epochs	96.20%	0.860
3	`sla_ta_e120_b1024`	AirResNet + SLA, 120 epochs	96.40%	0.863
4	`ta_e120_b1024_s1`	AirResNet, seed 1, 120 epochs	96.28%	0.857
5	`preact_ta_e200`	PreActResNet, 200 epochs	96.82%	0.884
6	`preact_sla_ta_e200_full`	PreActResNet + SLA, all 50k	(45k: 97.54%)	0.909
7	`preact_sla_percopy_e200`	PreActResNet + SLA + per-copy augmentation, 45k	97.66%	0.896
8	`preact_sla_cutmix_e300`	PreActResNet + SLA + CutMix, 300 epochs, 45k	97.70%	0.902
9	`preact_sla_ta_e300`	PreActResNet + SLA, 300 epochs, 45k	97.74%	0.904
10	`preact_sla_ta_e200`	PreActResNet + SLA, 200 epochs, 45k	97.54%	0.903
11	`preact_sla_ta_e300_full`	PreActResNet + SLA, 300 epochs, all 50k	(45k: 97.74%)	0.906
12	`preact_wide_sla_ta_e200`	Wide PreActResNet + SLA, 200 epochs, 45k	97.76%	0.898
13	`preact_wide_sla_ta_e200_full`	Wide PreActResNet + SLA, 200 epochs, all 50k	(45k: 97.76%)	0.901
14	`preact_sla_regmix_e200`	PreActResNet + SLA + RegMixup,	97.86%	0.907

#	Run	Model	Holdout	Kaggle public
		200 epochs, 45k		
15	`preact_sla_regmix_e200_full`	PreActResNet + SLA + RegMixup, 200 epochs, all 50k	(45k: 97.86%)	0.904
16	`preact_sla_regmix_e300_full`	PreActResNet + SLA + RegMixup, 300 epochs, all 50k (final)	(45k: 97.94%)	**0.912**

The holdout values of the SLA models are head + vote. One public image is approximately 0.001. All scores from 0.903 to 0.912 are in approximately one standard error. Thus, the public split cannot rank these models.

8. Final model

Item	Value
Architecture	Pre-activation ResNet, 3 stages of 4/4/3 basic blocks, widths 64/128/256, average pooling
Heads	10-class head + 40-class joint head (class × rotation)
Trainable parameters (torchsummary, forward pass)	4,696,394
Trainable parameters (all, with the 40-class head)	4,706,674
Training data	All 50,000 CIFAR-10 training images
Augmentation	TrivialAugmentWide, flip, 4-pixel translation, 12-pixel cutout
Loss	SLA+SD + RegMixup (Beta(20, 20), weight 1), label smoothing 0.1 on the 10-class head
Optimizer	Nesterov SGD, momentum 0.9, lr 0.2 at batch 256 + 256, weight decay 5e-4, 5 warmup epochs, cosine, 300 epochs
Prediction	Mean of softmax(head) and softmax(4-rotation vote), each with mirror + 1-pixel shift TTA
Holdout accuracy (same recipe on 45,000 images)	97.94%
Kaggle public accuracy	0.912
Weights	`models/regmix_e300_50k.pt` (submission), `models/regmix_e300_45k.pt` (holdout)
Submission	`submissions/submission_preact_sla_regmix_e300_50k.csv`

`notebooks/final_model.ipynb` loads the two weight files. It makes the submission again and compares it with the submitted CSV. The two files are equal, byte for byte. The notebook can also train the two models from zero.

9. Lessons

1. Do not select models with the public leaderboard. Use a holdout from the training data.
2. The architecture family has an effect on accuracy under shift. A fast airbench net was 1.4 points worse on Kaggle than a standard ResNet with the same holdout accuracy.

3. SLA (rotation labels) gave the largest gain for its cost: +0.4 to +0.7 points.
4. SAM, PAPA, distillation, longer schedules and anti-aliasing each gave +0.2 to +0.4 points on the plain ResNet. None of them was better than SLA.
5. RegMixup was the only method that added accuracy on top of SLA.
6. Calculate BN statistics on data that is similar to the training data. Clean-image statistics decreased the PAPA model by 1 point and the EMA model by 0.4 points.
7. Make long runs resumable. HPC jobs stop after 3 to 5 hours.
8. A wider and shallower network with equal holdout accuracy did not do better on the shifted test images.

10. Pipeline audit (2026-10-31)

We did a full review of the code, the data and our method. The review did not start a training run.

10.1 Result

We did not find an error that causes the gap between the holdout (97.6%) and Kaggle (0.90). These items are correct:

- All 50,000 training images and all 2,000 test images are 32 × 32 RGB PNG files. The cache is equal to the files.
- The labels and the IDs are equal to the CSV files. No test image is an exact copy of a training image.
- The training images and the test images get the same normalization.
- The SLA joint label is `class × 4 + rotation` in training and in prediction. The rotation order agrees.
- Augmentation occurs before the rotation. BN and bias parameters get no weight decay.
- All submission files have 2,000 rows, the correct ID order and labels 0 to 9.
- No code trains on the test images or labels them.

The test images are a little brighter and have more contrast than the training images:

Pixel statistic	Training	Test
Channel mean (R, G, B)	0.491, 0.482, 0.447	0.513, 0.498, 0.461
Channel std (R, G, B)	0.247, 0.243, 0.262	0.265, 0.262, 0.275
Spread of image brightness	0.125	0.144

Phase 2 shows that this difference costs a maximum of 0.4 points. Thus, it does not cause a 7-point gap. The line of Recht et al. (2018) gives approximately 92.8% at 97.7% holdout accuracy. Miller et al. (2021) show the same line for 1,060 models. Our 0.909 was approximately 1.6 points below the line. One standard error is approximately 0.9 points. Thus, we think that the gap comes from the shift in the hidden images, and not from an error.

10.2 Problems that we found and the changes

Problem	Effect	Change
We compared runs with the head only. But we submit head + vote.	SLA-300 gets +0.20 points on head + vote, not +0.12.	All comparisons from Phase 9 use head + vote.
The seed also selected the holdout split. Seed 0 and seed 1 have only 527 holdout images in common.	The seed spread of 0.2 points is not a correct measurement.	A separate split seed. All later runs use the seed-0 split.

Problem	Effect	Change
The EMA calculated BN statistics on upright images only. Training used 4 rotations.	The EMA result (–0.4 points) was too low.	BN statistics on the 4 rotated copies.
SAM did the second forward pass in training mode.	The BN statistics changed two times in each step.	The second pass does not change BN.
One submission script used the head only.	The submitted CSVs were correct.	One shared predictor (head + vote, weight 0.5).
A resumed run did not keep the TrivialAugment epoch or the random state.	A resumed run was not equal to a run without a stop.	The checkpoint keeps both.

10.3 Checks with saved models (2026-10-31)

Check (45k holdout, head + vote)	SLA, 200 epochs	SLA, 300 epochs	SLA + CutMix, 200 epochs
Current TTA (mirror + 2 diagonal 1-pixel shifts)	97.54%	97.74%	—
TTA with 4 axis 1-pixel shifts + mirror	97.54%	97.86%	—
TTA with 8 neighbour 1-pixel shifts + mirror	97.56%	97.76%	—
Live network	—	—	97.52%
EMA, BN on upright images (old method)	—	—	97.44%
EMA, BN on the 4 rotated copies	—	—	97.64%

The weight of the head and the vote in head + vote:

Run	Vote only	0.25 head	0.5 head (submitted)	0.75 head	Head only
SLA, 200 epochs	97.42%	97.50%	97.54%	97.64%	97.56%
SLA, 300 epochs	97.68%	97.84%	97.74%	97.70%	97.68%
SLA + CutMix, 300 epochs	97.64%	97.70%	97.70%	97.66%	97.66%

Decisions:

1. The other TTA versions do not give a gain on both runs. We keep the current TTA.
2. With the correct BN statistics, the EMA is 0.12 points better than the live network. This is less than our limit. Thus, we did not train a new EMA model.
3. The best weight is different for each run. The mean of the three runs is 97.68% (0.25), 97.66% (0.5) and 97.67% (0.75). We keep 0.5.
4. We do not use an ensemble of different networks. Two networks have 9.4M parameters. This is more than the 5M limit.

11. Repository cleanup (2026-11-04)

We removed the code of all experiments that are not part of the final model. The new code has five files: `src/prepare_data.py`, `src/model.py`, `src/data.py`, `src/train.py` and `src/predict.py`. The configuration and the training history of all runs stay in `results/runs/`.

Checks of the new code:

1. We trained the old code and the new code for 2 epochs on 800 images, on the CPU, with the same seed. The losses and the final weights are equal, bit for bit.
2. The new prediction code with `models/regmix_e300_50k.pt` gives a CSV that is equal to the submitted CSV, byte for byte. `models/regmix_e300_45k.pt` gives 97.94% on the holdout.